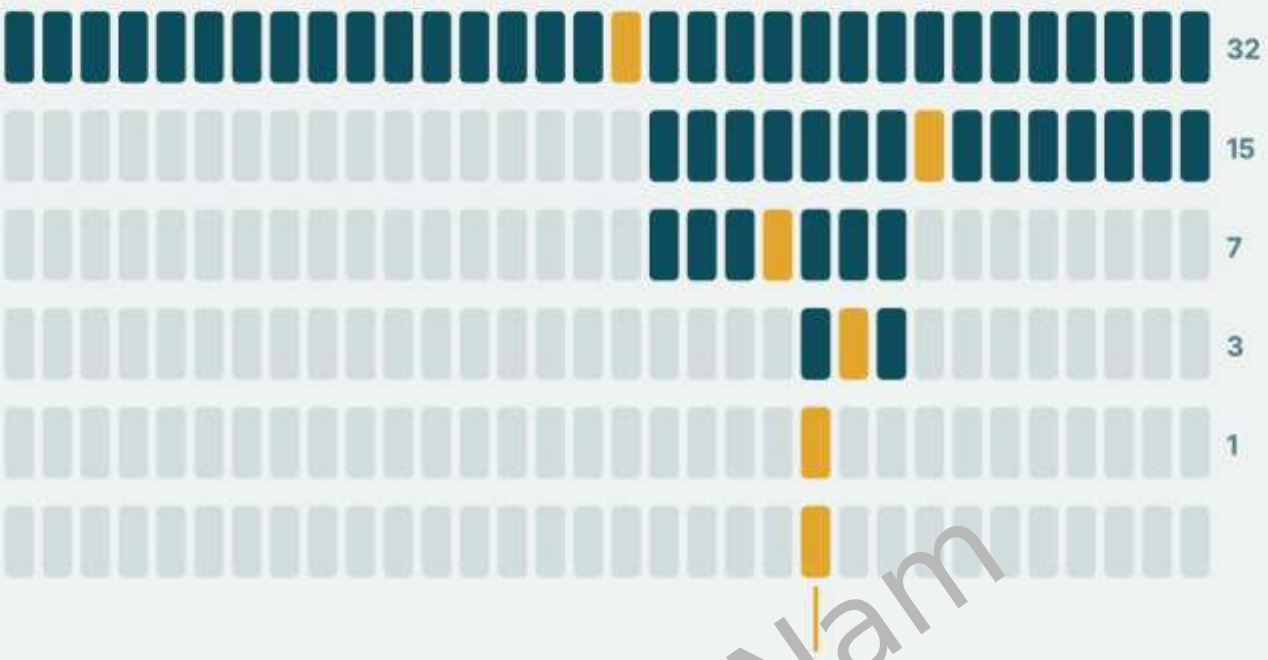


Tìm vị trí 21 trong 32 ô: mỗi hàng là một lần hỏi, ô tô vàng là mid, vùng còn nghi ngờ giảm ít nhất một nữa: 32 → 15 → 7 → 3 → 1 ô.

CHẶT NHỊ PHÂN

CƠ BẢN & NÂNG CAO

qua 24 bài Luogu 洛谷

```
while (lo < hi) {
  mid = lo + (hi - lo) / 2;
  if (check(mid)) hi = mid;
  else lo = mid + 1;
}
```

8 chương · 24 bài · 58 chương trình đã kiểm chứng và đo thời gian thật

1 Mảng đã sắp xếp · 2 Chặt nhị phân đáp án · 3 Max-min, min-max · 4 Tham lam & mô phỏng

5 Tiền tố & mảng hiệu · 6 Số thực · 7 Quy hoạch phân số · 8 Kết hợp đồ thị

LÊ THÀNH NAM

$\log_2 n$ bước

Lê Thành Nam

MỤC LỤC

MỤC LỤC 1
GIỚI THIỆU 3
Chương 1. Tìm kiếm nhị phân trên mảng đã sắp xếp 6
Bài 1. Tìm vị trí xuất hiện đầu tiên 7
Bài 2. Đếm cặp có hiệu bằng C 11
Bài 3. Nỗi lo nguyện vọng 14
Chương 2. Chặt nhị phân đáp án 17
Bài 4. Đốn cây 18
Bài 5. Cắt gỗ 22
Bài 6. Cắm biển chỉ đường 25
Chương 3. Lớn nhất của nhỏ nhất, nhỏ nhất của lớn nhất 28
Bài 7. Nhảy đá 29
Bài 8. Những chú bò hung dữ 33
Bài 9. Chia dãy thành đoạn II 36
Chương 4. Hàm kiểm tra: tham lam, mô phỏng và dựng nghiệm 39
Bài 10. Chép sách 40
Bài 11. Sân khấu cho bò nhảy 44
Bài 12. Quản lí thời gian 48
Chương 5. Hàm kiểm tra dùng tổng tiền tố và mảng hiệu 52
Bài 13. Người kiểm định thông minh 53
Bài 14. Mượn phòng họ 57
Bài 15. Máy tự giải [illegible] 60
Chương 6. Chặt nhị phân trên số thực 63
Bài 16. Giải phương trình bậc ba 64
Bài 17. Cắt dây 67
Bài 18. Thiết bị của Kotori 70
Chương 7. Trung bình, tỉ lệ và quy hoạch phân số 73
Bài 19. Trung bình lớn nhất 74
Bài 20. Buổi biểu diễn tài năng 78
Bài 21. Giao hàng 81
Chương 8. Chặt nhị phân kết hợp đồ thị 84
Bài 22. Giải cứu 85
Bài 23. Đường về Orgrimmar 89
Bài 24. Ám sát đại sứ 93
PHỤ LỤC 97

Lê Thành Nam

Lê Thành Nam

GIỚI THIỆU

Cuốn sách này dành cho ai

Sách gồm 24 bài chặt nhị phân chọn từ kho đề Luogu (洛谷, luogu.com.cn), nền tảng luyện thi tin học lớn nhất Trung Quốc. Các bài được xếp thành 8 chương, đi từ tìm kiếm nhị phân trên mảng đã sắp xếp đến chặt nhị phân đáp án, các dạng "lớn nhất của nhỏ nhất", hàm kiểm tra dùng tham lam, mô phỏng, tổng tiền tố và mảng hiệu, chặt nhị phân trên số thực, quy hoạch phân số, và chặt nhị phân kết hợp các thuật toán đồ thị.

Phần lớn các bài là đề thi chính thức (NOIP, USACO, COCI, các kì thi chọn đội tuyển tỉnh của Trung Quốc như SHOI, TJOI) hoặc các bài kinh điển mà học sinh Trung Quốc nào cũng luyện. Bài dễ nhất phù hợp học sinh mới học tìm kiếm nhị phân; bài khó nhất đòi hỏi kết hợp chặt nhị phân với quy hoạch động hoặc đường đi ngắn nhất.

Mỗi bài gồm hai phần. **Đề bài** được viết lại bằng tiếng Việt, giữ đúng yêu cầu, ràng buộc, định dạng vào/ra và ví dụ của đề gốc, kèm tên gốc tiếng Trung và bảng subtask do sách bổ sung. **Hướng dẫn giải** đi qua nhiều tầng: lời giải ngây thơ, các tầng cải tiến (kể cả những tầng "nhanh nhưng sai"), rồi mới đến lời giải tối ưu. Mỗi tầng đều có code chạy được, thời gian chạy đo thật và hình minh hoạ.

> **Về cách viết đề**
>
> Đề trong sách không phải bản dịch từng câu. Mỗi đề được viết lại theo cách diễn tự nhiên của người Việt, bỏ những chi tiết kể chuyện không ảnh hưởng đến lời giải. Mọi điều kiện, giới hạn, định dạng vào/ra và test ví dụ được giữ nguyên; mọi ví dụ đã được đối chiếu bằng cách chạy lời giải. Giới hạn thời gian ghi trong sách là giới hạn sách dùng để chấm các subtask, có thể khác giới hạn trên Luogu.

Cách giải

Hướng dẫn giải của mỗi bài chia thành các phần có dải màu riêng; hầu hết các bước đều có hình minh hoạ.

Phần	Nội dung
HIỂU ĐỀ	Vài ví dụ các trường hợp để hiểu chính xác yêu cầu; nêu các nhận xét đầu tiên, thường là tính đơn điệu
NGÂY THƠ	Tầng 1: ý tưởng tự nhiên nhất (thường là thử từng giá trị đáp án), code và độ phức tạp
BẾ TẮC	Thời gian chạy đo thật của tầng 1, chỉ ra nguyên nhân chậm, kết thúc bằng một *câu hỏi then chốt*
CẢI TIẾN	Các tầng trung gian, có cả những tầng "nhanh nhưng sai": lặp vô hạn, tràn số, sai cận, quên sắp xếp, làm tròn sai
LÓE SÁNG	Nhận xét quyết định: tính đơn điệu, hàm kiểm tra, cách đổi câu hỏi tối ưu thành câu hỏi có/không
TỐI ƯU	Code tối ưu, độ phức tạp và bảng so sánh tất cả các tầng trên mọi subtask
LƯU Ý	Những chỗ dễ sai và phần mở rộng

Mỗi bài có ít nhất một hộp **Thiết kế lời giải** màu vàng, tóm tắt hàm kiểm tra, khoảng tìm và khuôn chặt nhị phân được dùng.

Thời gian chạy trong sách được đo thật bằng trình biên dịch g++ 13 (tuỳ chọn -O2, chuẩn C++17) trên máy chủ Linux, tính cả thời gian đọc và ghi dữ liệu. Trên máy chấm Luogu các con số có thể khác đôi chút, nhưng tỉ lệ giữa các tầng là như nhau.

Subtask và các tầng lời giải

Mỗi bài có một bảng subtask: subtask đầu dành cho lời giải ngây thơ, subtask cuối là ràng buộc gốc của Luogu (hoặc mở rộng, có ghi rõ). Một số bài có subtask "test đặc biệt", được thiết kế để bắt một lỗi thường gặp (làm tròn sai, nhân sai số thực…). Cuối mỗi bài có bảng so sánh: mỗi tầng chạy trên test lớn nhất của từng subtask, ghi thời gian thật và số điểm đạt được.

Lê Thành Nam

Sử dụng sách

Với mỗi bài, nên đọc đề, tự nghĩ ít nhất 20–30 phút rồi mới đọc hướng dẫn, và dừng lại ở mỗi "Câu hỏi then chốt" để tự trả lời trước khi đọc tiếp. Mọi code trong sách đã được chạy với ví dụ của đề và đối chiếu chéo với nhau trên 120 test ngẫu nhiên mỗi bài (trừ những tầng được cố ý trình bày như một lời giải sai).

Nộp bài trên Luogu

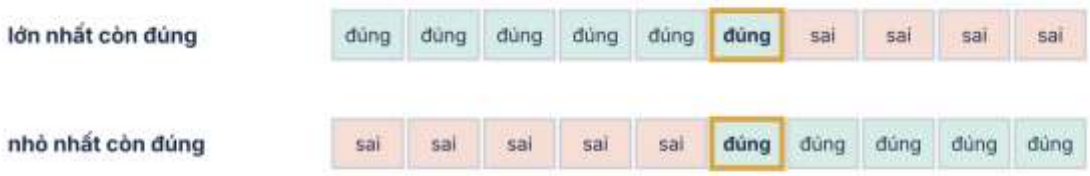
- Đăng kí tài khoản tại luogu.com.cn (giao diện có thể chuyển sang tiếng Anh).
- Mở đề theo mã bài (ví dụ P2249) bằng đường dẫn trong sách, chọn 提交答案 (nộp bài), chọn ngôn ngữ C++14 hoặc C++17 (bật O2 nếu muốn), dán code và nộp.
- Chương trình đọc từ bàn phím (stdin) và ghi ra màn hình (stdout).
- Kết quả: AC (đúng), WA (sai đáp án), TLE (quá thời gian), MLE (quá bộ nhớ), RE (lỗi khi chạy).

Bốn bước tư duy trước một bài chặt nhị phân



Bốn bước xây dựng lời giải một bài chặt nhị phân

- **Bước 1. Đọc ràng buộc.** Đáp án lớn tới đâu (quyết định khoảng tìm và kiểu dữ liệu), hàm kiểm tra được phép tốn bao nhiêu (thường O(n) hoặc O(n log n)).
- **Bước 2. Viết hàm kiểm tra.** Cho trước một giá trị x, trả lời câu hỏi có/không: "làm được với x không?". Thường dễ hơn nhiều so với tìm thẳng đáp án.
- **Bước 3. Chứng minh đơn điệu.** Nếu x làm được thì x + 1 (hoặc x − 1) cũng làm được. Không có bước này thì chặt nhị phân sai.
- **Bước 4. Chọn khuôn và khoảng tìm.** Tìm lớn nhất hay nhỏ nhất còn đúng; [lo, hi] phải chắc chắn chứa đáp án.



Hai khuôn của sách. Ô viền vàng là đáp án cần tìm.

Khoảng giá trị đáp án	Số lần kiểm tra	Ghi chú
10^6	≈ 20	vị trí trong mảng, số lượng
10^9 – $2 \cdot 10^9$	≈ 31	độ dài, chiều cao, khoảng cách: cẩn thận tràn int khi tính mid
10^{18}	≈ 60	tổng lớn: long long, mid = lo + (hi − lo)/2
số thực	60 – 100 (cố định)	không so sánh với eps; nếu đáp án có độ chính xác cố định, đổi sang số nguyên

Lê Thanh Nam

Bố cục sách

Chương	Chủ đề	Bài (mã Luogu)
1	Tìm kiếm nhị phân trên mảng đã sắp xếp	P2249, P1102, P1678
2	Chặt nhị phân đáp án	P1873, P2440, P3853
3	Lớn nhất của nhỏ nhất, nhỏ nhất của lớn nhất	P2678, P1824, P1182
4	Hàm kiểm tra: tham lam, mô phỏng và dựng nghiệm	P1281, P3611, P2920
5	Hàm kiểm tra dùng tổng tiền tố và mảng hiệu	P1314, P1083, P4343
6	Chặt nhị phân trên số thực	P1024, P1577, P3743
7	Trung bình, tỉ lệ và quy hoạch phân số	P1404, P4377, P1542
8	Chặt nhị phân kết hợp đồ thị	P1396, P1462, P1902

Lê Thành Nam

Chương 1. Tìm kiếm nhị phân trên mảng đã sắp xếp

Trọng tâm chương

Chương mở đầu với tình huống quen thuộc nhất: tìm trong một mảng đã sắp xếp. Trọng tâm là viết chặt nhị phân **theo bất biến** để không bao giờ lặp vô hạn hay lệch một đơn vị, và dùng thành thạo cặp **lower_bound / upper_bound** để đếm, tìm phần tử gần nhất.

Kiến thức cần nhớ

- Mọi bài chặt nhị phân là tìm **ranh giới** của một điều kiện đơn điệu: sai, ..., sai, đúng, ..., đúng.
- Bất biến: mọi vị trí < l là "sai", mọi vị trí ≥ r là "đúng"; dừng khi l = r.
- Gán r = mid thì mid làm tròn xuống; gán l = mid thì mid làm tròn lên.
- lower_bound(x): vị trí đầu tiên ≥ x; upper_bound(x): vị trí đầu tiên > x; số phần tử bằng x = hiệu của hai cận.
- Phần tử gần x nhất trong tập đã sắp xếp: chỉ cần xét hai láng giềng của lower_bound(x).

Các bài trong chương

Bài	Tên bài	Mã Luogu	Nguồn	Số tầng
1	Tìm vị trí xuất hiện đầu tiên 【深基 13.例 1】查找	P2249	Luogu (bài mẫu về tìm kiếm nhị phân)	3
2	Đếm cặp có hiệu bằng C A-B 数对	P1102	Luogu (bài kinh điển về đếm bằng tìm kiếm nhị phân)	3
3	Nỗi lo nguyện vọng 烦恼的高考志愿	P1678	Luogu (bài kinh điển về tìm phần tử gần nhất)	2

Lê Thành Nam

Bài 1. Tìm vị trí xuất hiện đầu tiên

Luogu P2249 – 【深基 13.例 1】查找

Nguồn: Luogu (bài mẫu về tìm kiếm nhị phân) · Giới hạn thời gian sách dùng: 1,0 giây · Bộ nhớ: 128 MB
Đề gốc:https://www.luogu.com.cn/problem/P2249

Đề bài

Cho dãy n số nguyên **không giảm** $a_1, \ldots, a_n$ và m câu hỏi. Mỗi câu hỏi cho một số q; hãy in vị trí **đầu tiên** của q trong dãy (đánh số từ 1), hoặc −1 nếu q không có trong dãy.

Dữ liệu vào

Dòng đầu chứa n và m. Dòng thứ hai chứa n số a_i. Dòng thứ ba chứa m số q.

Kết quả

Một dòng gồm m số, cách nhau một dấu cách.

Ràng buộc

- $n \le 10^6$; $m \le 10^5$; $0 \le a_i, q \le 10^9$.

Ví dụ

Dữ liệu vào	Kết quả
11 3 1 3 3 3 5 7 9 11 13 15 15 1 3 6	1 2 -1

Subtask

Các subtask dưới đây do sách bổ sung để luyện từng mức thuật toán; subtask cuối cùng là ràng buộc của đề gốc (hoặc mở rộng hơn đề gốc). Giới hạn thời gian 1,0 giây cho mọi subtask.

Subtask	Ràng buộc	Điểm
ST1	$n, m \le 1000$	30%
ST2	$n \le 10^6$, $m \le 10^5$ (gốc)	70%

Lê Thành Nam

Hướng dẫn giải

HIỂU ĐỀ Hiểu đúng đề bài

Số 3 xuất hiện ở các vị trí 2, 3, 4; đáp án là 2 (vị trí đầu tiên). Số 6 không có trong dãy nên in −1.

Câu hỏi "vị trí đầu tiên của q" tương đương với: **tìm i nhỏ nhất sao cho $a_i \ge q$**, rồi kiểm tra a_i có bằng q không. Cách diễn đạt thứ hai có lợi vì điều kiện "$a_i \ge q$" là **đơn điệu**: sai, sai, …, sai, đúng, đúng, …, đúng. Mọi bài chặt nhị phân đều quy về việc tìm **ranh giới** giữa phần sai và phần đúng như vậy.

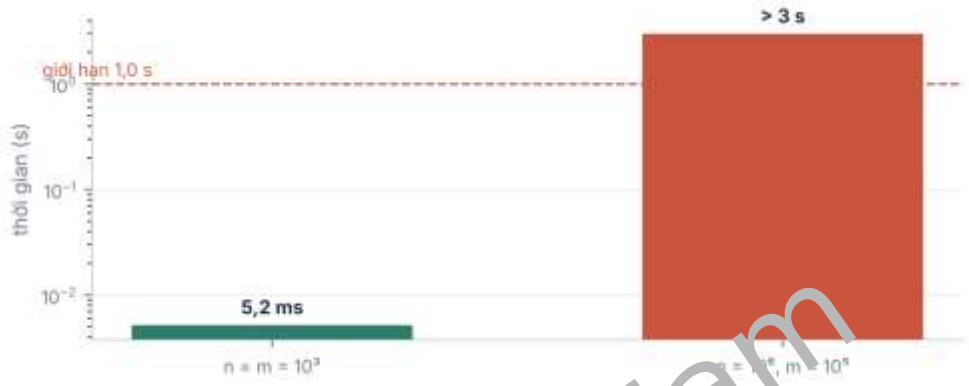
NGÂY THƠ Ý tưởng ngây thơ: quét từ đầu

Code. Tầng 1 – Quét tuyến tính cho mỗi truy vấn

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n, m;
    scanf("%d %d", &n, &m);
```

```
    vector<int> a(n);
    for (int &x : a) scanf("%d", &x);
    for (int k = 0; k < m; k++) {
        int q, ans = -1;
        scanf("%d", &q);
        for (int i = 0; i < n; i++)          // quét từ đầu dãy
            if (a[i] == q) { ans = i + 1; break; }
        printf("%d%c", ans, k == m - 1 ? '\n' : ' ');
    }
}
```

BẾ TẮC Bế tắc: $n \cdot m = 10^{11}$



Hình 1.1. Tầng 1 trên các nhóm test (thời gian đo thật, trục log)

Câu hỏi then chốt

Dãy đã được sắp xếp. Nếu nhìn vào phần tử ở giữa đoạn đang xét và thấy nó nhỏ hơn q, ta loại được những phần tử nào?

LÓE SÁNG Lóe sáng: mỗi lần hỏi loại một nửa

Nếu a[mid] < q thì mọi phần tử bên trái mid cũng nhỏ hơn q: loại cả nửa trái (kể cả mid). Nếu a[mid] ≥ q thì mid **có thể** là đáp án, còn mọi phần tử bên phải mid chắc chắn không phải (không phải vị trí đầu tiên): loại nửa phải, giữ mid.

Mỗi bước đoạn đang xét giảm một nửa, nên chỉ cần khoảng $\log_2 10^6 \approx 20$ bước.

CẢI TIẾN 1 Cải tiến 1 (sai): chặt nhị phân "tự nhiên"

Một cách viết rất hay gặp:

Code. Tầng 2 – Chặt nhị phân viết sai (l = mid)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n, m;
    scanf("%d %d", &n, &m);
    vector<int> a(n);
    for (int &x : a) scanf("%d", &x);
    for (int k = 0; k < m; k++) {
        int q;
        scanf("%d", &q);
        int l = 0, r = n - 1;
        while (l < r) {                      // SAI: mid làm tròn xuống nhưng gán l = mid
            int mid = (l + r) / 2;
            if (a[mid] < q) l = mid;         // khi r = l + 1 và a[l] < q: lặp vô hạn
            else r = mid;
        }
        int ans = (a[l] == q) ? l + 1 : -1;
        printf("%d%c", ans, k == m - 1 ? '\n' : ' ');
    }
}
```

Lê Thành Nam

Khi đoạn chỉ còn hai phần tử (r = l + 1) và a[l] < q, ta có mid = l, và lệnh l = mid **không làm đoạn nhỏ lại**. Chương trình lặp vô hạn, ngay cả trên ví dụ của đề.

LÓE SÁNG Lóe sáng: viết theo bất biến

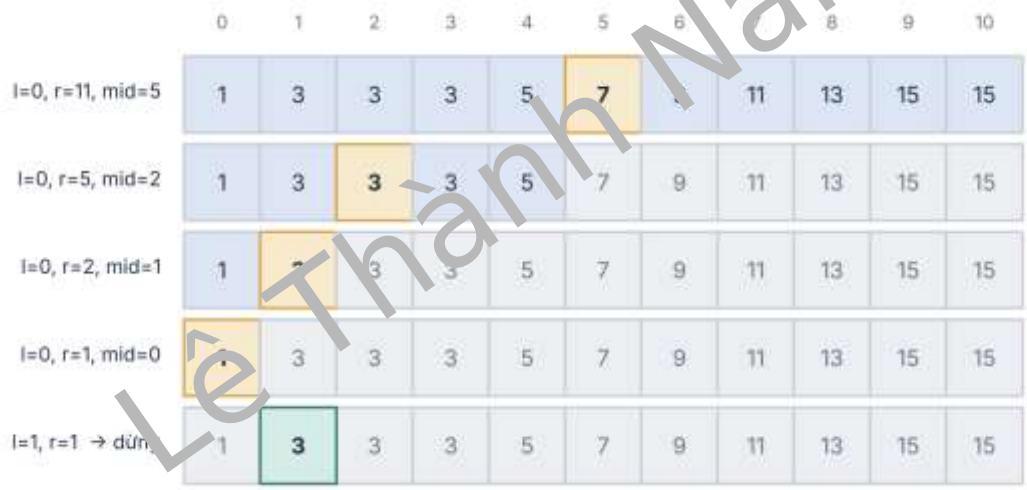
Để không bao giờ phải "thử sửa" các dấu +1/−1, hãy cố định một **bất biến** và viết code để giữ nó:

- mọi vị trí < l có a[i] < q;
- mọi vị trí ≥ r có a[i] ≥ q;
- đáp án nằm trong [l, r]; dừng khi l = r.



Hình 1.2. *Bất biến: phần bên trái l chắc chắn "sai", phần từ r trở đi chắc chắn "đúng", chỉ [l, r) là chưa biết.*

Từ bất biến suy ra ngay hai lệnh cập nhật: a[mid] ≥ q thì r = mid; ngược lại l = mid + 1. Vì mid < r luôn đúng khi l < r (mid làm tròn xuống), mỗi bước đoạn [l, r) đều ngắn đi ít nhất 1: không bao giờ lặp vô hạn.



Hình 1.3. *Chặt nhị phân tìm q = 3 trên ví dụ. Ô vàng là mid, ô xanh lá là kết quả: vị trí 1 (tức vị trí 2 khi đánh số từ 1).*

TỐI ƯU Lời giải tối ưu

Code. Tầng 3 – Chặt nhị phân với bất biến rõ ràng

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n, m;
    scanf("%d %d", &n, &m);
    vector<int> a(n);
    for (int &x : a) scanf("%d", &x);
    for (int k = 0; k < m; k++) {
        int q;
        scanf("%d", &q);
        // tìm vị trí đầu tiên có a[i] >= q trong [0, n]; bất biến: a[l-1] < q <= a[r]
        int l = 0, r = n;
        while (l < r) {
            int mid = l + (r - l) / 2;
            if (a[mid] >= q) r = mid;        // mid có thể là đáp án: giữ lại
            else l = mid + 1;                // mid chắc chắn không phải: bỏ
        }
        int ans = (l < n && a[l] == q) ? l + 1 : -1;
        printf("%d%c", ans, k == m - 1 ? '\n' : ' ');
    }
```

Lê Thành Nam

```
}
```

Mỗi truy vấn O(log n). Thư viện C++ có sẵn hàm làm đúng việc này: lower_bound(a.begin(), a.end(), q) trả về vị trí đầu tiên có giá trị ≥ q; còn upper_bound trả về vị trí đầu tiên có giá trị > q.

Tầng	Ý tưởng	Độ phức tạp	$n = m = 10^3$ (ST1)	$n = 10^6, m = 10^5$ (ST2)	Điểm
1	Quét tuyến tính cho mỗi truy vấn	O(nm)	5,2 ms	quá giờ	**30**
2	Chặt nhị phân viết sai (l = mid)	lặp vô hạn	quá giờ	quá giờ	**0**
3	Chặt nhị phân với bất biến rõ ràng	O(m log n)	6,6 ms	0,11 s	**100**

Thời gian đo thật trên test lớn nhất của từng subtask; giới hạn 1,0 giây. Ô đỏ: quá giờ, sai hoặc quá bộ nhớ. Cột "Điểm" cộng phần trăm của các subtask mà tầng đó vượt qua.

LƯU Ý Những chỗ dễ sai

- Đáp án có thể là n (mọi phần tử đều nhỏ hơn q), nên khởi tạo r = n chứ không phải n − 1, và kiểm tra l < n trước khi đọc a[l].
- Viết mid = l + (r − l) / 2 để tránh tràn số khi l, r lớn (quan trọng khi chặt trên giá trị tới $2 \cdot 10^9$).
- Quy tắc ghi nhớ: **gán l = mid thì mid phải làm tròn lên** ((l + r + 1) / 2); **gán r = mid thì mid làm tròn xuống**. Viết sai cặp này là nguyên nhân số một của lặp vô hạn.
- Với 10^6 số đầu vào, scanf là đủ nhanh; cin cần ios::sync_with_stdio(false).

Lê Thành Nam

Bài 2. Đếm cặp có hiệu bằng C

Luogu P1102 – A-B 数对

Nguồn: Luogu (bài kinh điển về đếm bằng tìm kiếm nhị phân) · Giới hạn thời gian sách dùng: 1,0 giây · Bộ nhớ: 128 MB
Đề gốc:https://www.luogu.com.cn/problem/P1102

Đề bài

Cho dãy N số nguyên không âm và một số nguyên dương C. Hãy đếm số cặp vị trí (i, j) sao cho $a_i - a_j = C$. Hai cặp có cùng giá trị nhưng khác vị trí được tính là khác nhau.

Dữ liệu vào

Dòng đầu chứa N và C. Dòng thứ hai chứa N số a_i.

Kết quả

Một số nguyên là số cặp.

Ràng buộc

- $1 \le N \le 2 \cdot 10^5$; $0 \le a_i < 2^{30}$; $1 \le C < 2^{30}$.

Ví dụ

Dữ liệu vào	Kết quả
4 1 1 1 2 3	3

Subtask

Các subtask dưới đây do sách bổ sung để luyện từng mức thuật toán; subtask cuối cùng là ràng buộc của đề gốc (hoặc mở rộng hơn đề gốc). Giới hạn thời gian 1,0 giây cho mọi subtask.

Subtask	Ràng buộc	Điểm
ST1	$N \le 3000$	40%
ST2	$N \le 2 \cdot 10^5$ (gốc)	60%

Lê Thành Nam

Hướng dẫn giải

HIỂU ĐỀ Hiểu đúng đề bài

Ở ví dụ (C = 1), các cặp (A, B) là (2, 1) với hai số 1 khác nhau, và (3, 2): tổng cộng 3. Với mỗi phần tử đóng vai trò A, cần đếm **có bao nhiêu phần tử bằng A − C**.

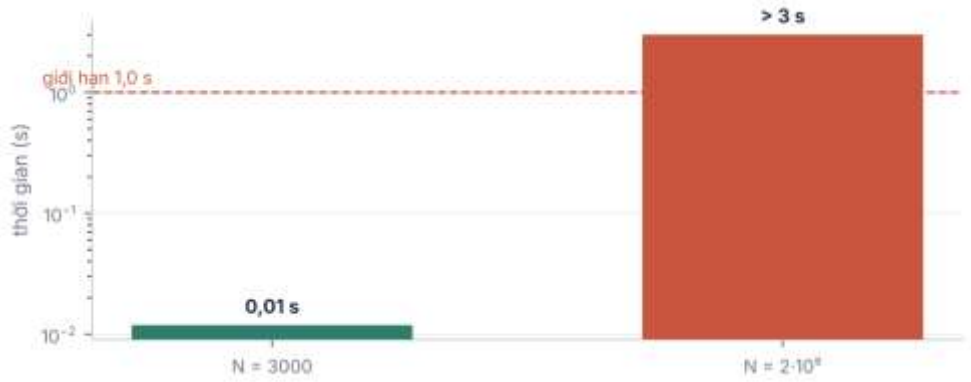
NGÂY THƠ Ý tưởng ngây thơ: thử mọi cặp

Code. Tầng 1 – Thử mọi cặp

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n; long long C;
    scanf("%d %lld", &n, &C);
    vector<long long> a(n);
    for (auto &x : a) scanf("%lld", &x);
    long long cnt = 0;
    for (int i = 0; i < n; i++)              // thử mọi cặp vị trí (A ở i, B ở j)
        for (int j = 0; j < n; j++)
```

```
            if (a[i] - a[j] == C) cnt++;
    printf("%lld\n", cnt);
}
```

BẾ TẮC Bế tắc: $N^2 = 4 \cdot 10^{10}$



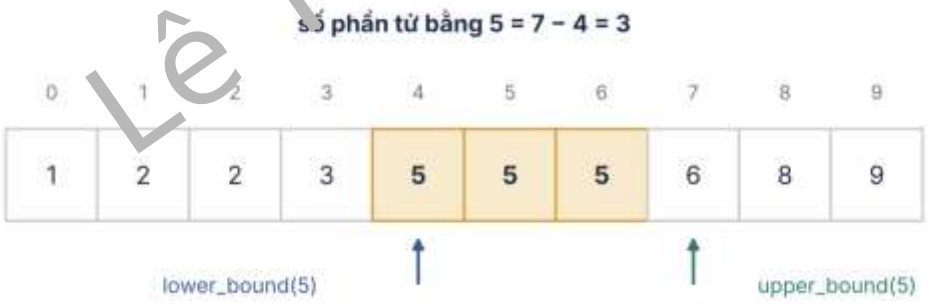
Hình 2.1. *Tầng 1 trên các nhóm test (thời gian đo thật, trục log)*

Câu hỏi then chốt

Trong một dãy **đã sắp xếp**, các phần tử bằng x nằm ở đâu? Làm sao đếm chúng mà không duyệt qua từng phần tử?

LÓE SÁNG Lóe sáng: đếm = hiệu của hai cận

Sau khi sắp xếp, các phần tử bằng x tạo thành **một đoạn liên tiếp**. Đoạn đó bắt đầu tại lower_bound(x) (vị trí đầu tiên ≥ x) và kết thúc ngay trước upper_bound(x) (vị trí đầu tiên > x). Vậy số phần tử bằng x là upper_bound(x) − lower_bound(x).



Hình 2.2. *Trong dãy đã sắp xếp, các số 5 nằm liền nhau; hai cận cho biết đoạn đó bắt đầu và kết thúc ở đâu.*

Thiết kế lời giải	
Bước 1	sắp xếp dãy, O(N log N)
Bước 2	với mỗi i, cộng upper_bound(a_i − C) − lower_bound(a_i − C), O(log N)
Đáp án	tổng ở bước 2

CẢI TIẾN 1 Cải tiến 1 (sai): quên ước lượng đáp án

Code. Tầng 2 – Sắp xếp + đếm bằng hai cận, biến đếm int

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n; long long C;
```

Lê Thanh Nam

```
    scanf("%d %lld", &n, &C);
    vector<long long> a(n);
    for (auto &x : a) scanf("%lld", &x);
    sort(a.begin(), a.end());
    int cnt = 0;                              // SAI: số cặp có thể tới n^2/4 = 10^10
    for (int i = 0; i < n; i++) {
        // số phần tử bằng a[i] - C = (vị trí đầu tiên > x) - (vị trí đầu tiên >= x)
        long long x = a[i] - C;
        cnt += upper_bound(a.begin(), a.end(), x) - lower_bound(a.begin(), a.end(), x);
    }
    printf("%d\n", cnt);
}
```

Ở ST2, nửa dãy bằng 5 và nửa dãy bằng 12 (C = 7): mỗi số 12 ghép được với 10^5 số 5, nên có 10^{10} cặp, vượt quá int.

TỐI ƯU Lời giải tối ưu

Code. Tầng 3 – Sắp xếp + đếm bằng hai cận, long long

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n; long long C;
    scanf("%d %lld", &n, &C);
    vector<long long> a(n);
    for (auto &x : a) scanf("%lld", &x);
    sort(a.begin(), a.end());
    long long cnt = 0;                        // số cặp có thể tới n^2/4 = 10^10
    for (int i = 0; i < n; i++) {
        // số phần tử bằng a[i] - C = (vị trí đầu tiên > x) - (vị trí đầu tiên >= x)
        long long x = a[i] - C;
        cnt += upper_bound(a.begin(), a.end(), x) - lower_bound(a.begin(), a.end(), x);
    }
    printf("%lld\n", cnt);
}
```

Tầng	Ý tưởng	Độ phức tạp	N = 3000 (ST1)	N = 2·10⁵ (ST2)	Điểm
1	Thử mọi cặp	O(N^2)	0,01 s	quá giờ	**40**
2	Sắp xếp + đếm bằng hai cận, biến đếm int	O(N log N)	4,1 ms	sai	**40**
3	Sắp xếp + đếm bằng hai cận, long long	O(N log N)	4,1 ms	0,02 s	**100**

Thời gian đo thật trên test lớn nhất của từng subtask; giới hạn 1,0 giây. Ô đỏ: quá giờ, sai hoặc quá bộ nhớ. Cột "Điểm" cộng phần trăm của các subtask mà tầng đó vượt qua.

Lê Thành Nam

LƯU Ý Những chỗ dễ sai

- **Đáp án có thể tới $N^2/4$**: luôn ước lượng trước khi chọn kiểu dữ liệu.
- Cũng có thể dùng hai con trỏ trên dãy đã sắp xếp (O(N) sau khi sắp xếp), hoặc map đếm tần suất. Cách dùng hai cận là cách gọn và ít lỗi nhất.
- Hai cận lower_bound / upper_bound là "con dao đa năng": đếm số phần tử trong đoạn giá trị [x, y] là upper_bound(y) − lower_bound(x).

Bài 3. Nỗi lo nguyện vọng

Luogu P1678 – 烦恼的高考志愿

Nguồn: Luogu (bài kinh điển về tìm phần tử gần nhất) · Giới hạn thời gian sách dùng: 1,0 giây · Bộ nhớ: 128 MB
Đề gốc:https://www.luogu.com.cn/problem/P1678

Đề bài

Có m trường, trường i có điểm chuẩn dự kiến a_i. Có n học sinh, học sinh j ước lượng mình được b_j điểm. Mỗi học sinh chọn trường có điểm chuẩn **gần với điểm của mình nhất** (độ "không hài lòng" là $|a_i - b_j|$). Nhiều học sinh có thể chọn cùng một trường.

Hãy tính tổng độ không hài lòng nhỏ nhất của tất cả học sinh.

Dữ liệu vào

Dòng đầu chứa m và n. Dòng thứ hai chứa m số a_i. Dòng thứ ba chứa n số b_j.

Kết quả

Một số nguyên là tổng nhỏ nhất.

Ràng buộc

- $1 \le n, m \le 10^5$; $0 \le a_i, b_j \le 10^6$.

Ví dụ

Dữ liệu vào	Kết quả
4 3 513 598 567 689 500 600 550	32

Subtask

Các subtask dưới đây do sách bổ sung để luyện từng mức thuật toán; subtask cuối cùng là ràng buộc của đề gốc (hoặc mở rộng hơn đề gốc). Giới hạn thời gian 1,0 giây cho mọi subtask.

Subtask	Ràng buộc	Điểm
ST1	$m, n \le 1000$	30%
ST2	$m, n \le 10^5$ (gốc)	70%

Hướng dẫn giải

HIỂU ĐỀ Hiểu đúng đề bài

Các học sinh độc lập với nhau: mỗi bạn chỉ cần tìm trường có điểm gần mình nhất. Ở ví dụ: 500 → 513 (13), 600 → 598 (2), 550 → 567 (17), tổng 32.

Lê Thành Nam



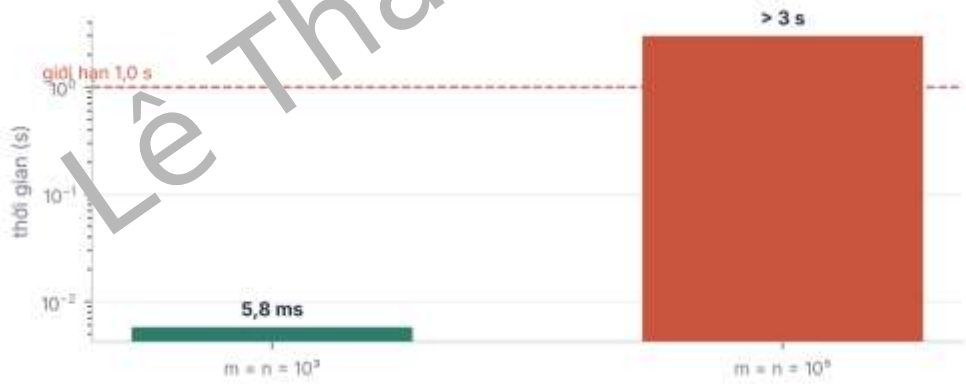
Hình 3.1. *Mỗi học sinh chọn trường gần nhất trên trục số.*

NGÂY THƠ Ý tưởng ngây thơ: duyệt mọi trường

Code. Tầng 1 – Mỗi học sinh duyệt mọi trường

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int m, n;
    scanf("%d %d", &m, &n);
    vector<int> a(m), b(n);
    for (int &x : a) scanf("%d", &x);
    for (int &x : b) scanf("%d", &x);
    long long sum = 0;
    for (int s : b) {                        // với mỗi học sinh, duyệt mọi trườn
        int best = INT_MAX;
        for (int x : a) best = min(best, abs(x - s));
        sum += best;
    }
    printf("%lld\n", sum);
}
```

BẾ TẮC Bế tắc: 10^{10} phép so sánh



Hình 3.2. *Tầng 1 trên các nhóm test (thời gian đo thật, trục log)*

> **Câu hỏi then chốt**
>
> Trên trục số, trường gần học sinh s nhất chắc chắn nằm trong số những trường nào?

LÓE SÁNG Lóe sáng: chỉ cần hai láng giềng

Sắp xếp điểm các trường. Gọi p = lower_bound(s): trường đầu tiên có điểm ≥ s. Trường gần nhất chỉ có thể là **a[p]** (láng giềng bên phải) hoặc **a[p − 1]** (láng giềng bên trái): mọi trường khác đều xa hơn một trong hai trường này.

Thiết kế lời giải	
Bước 1	sắp xếp a

Lê Thành Nam